theory *Week_3_sol*
imports *Main*
begin

King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Term 1

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 3

About these solutions. Every Isabelle proof method (e.g. *rule*, *simp*), Isar keyword (e.g. *have*, *also*), attribute (e.g. *OF*, *of*) and method modifier (e.g. *arbitrary:*, *simp add:*) is explained in a short comment, marked —, the first time it appears in the solutions, in sheet order from Week 1. Later uses are not explained again, so if you meet an unfamiliar construct, look for its first use in an earlier solution. Each proof is also labelled with its style; the module's preferred style is declarative (Isar), where every step states what it proves.

Exercise 3.1 Fold function

The fold function is a very generic function, that can be used to express multiple other interesting functions over lists.

Have a look at Isabelle/HOL's standard function *fold*, defined as follows:

fun *fold*::$('a \Rightarrow 'b \Rightarrow 'b) \Rightarrow 'a\ list \Rightarrow 'b \Rightarrow 'b$ **where**
fold f [] *acc* = *acc*
| *fold f* (*x* #*xs*) *acc* = *fold f xs* (*f x acc*)

Write a function to compute the sum of the elements of a list. Define two versions, one direct recursive definition, and one using fold. Show that both are equal.

Hint: use automation!

fun *list_sum* :: *nat list* ⇒ *nat*

where
list_sum [] = *0*

| *list_sum* (*x*#*xs*) = *x* + *list_sum xs*

definition *list_sum′* :: *nat list* ⇒ *nat*

  where *list_sum′ xs* ≡ *fold* (+) *xs 0*

These are three proofs of the key fact relating *fold* and *list_sum*, in different styles. The first is an apply script using only substitutions, the second is a declarative Isar proof with equational reasoning, and the third uses automation. In this module we prefer the declarative style: every step states what it proves, so the proof can be read and checked without running Isabelle, like a pen-and-paper proof.

lemma *fold* (+) *xs acc* = *acc* + *list_sum xs*
proof(*induction xs arbitrary*: *acc*)
  — Style: apply-style (procedural), shown only for contrast; generalised list induction (new). Each case is a sequence of *subst* commands, and the intermediate goals are visible only in Isabelle.
  case *Nil*
  show *?case*
    apply(*subst fold.simps*(*1*))
    apply(*subst list_sum.simps*(*1*))
    apply(*subst add_0_right*)
    ..
next
  case (*Cons a xs*)
  show *?case*
    apply(*subst fold.simps*(*2*))
    apply(*subst Cons.IH*)
    apply(*subst list_sum.simps*(*2*))
    apply(*subst add.assoc*[*symmetric*])
    apply(*subst add.commute*)
    ..
qed

lemma *fold* (+) *xs acc* = *acc* + *list_sum xs*
proof(*induction xs arbitrary*: *acc*)
  — Style: declarative (Isar), the preferred style; the same *subst* steps as above, but each intermediate equation is stated explicitly with *also have* … *finally*.
  case *Nil*
  have *fold* (+) [] *acc* = *acc*
    apply(*subst fold.simps*(*1*))
    ..
  also have ... = *acc* + *0*
    apply(*subst add_0_right*)
    ..
  also have ... = *acc* + *list_sum* []
    apply(*subst list_sum.simps*(*1*))
    ..

```
  finally show ?case
    .
next
  case (Cons a xs)
  have fold (+) (a#xs) acc = fold (+) xs (a+acc)
    apply(subst fold.simps(2))
    ..
  also have ... = a + acc + list_sum xs
    apply(subst Cons.IH)
    ..
  also have ... = acc + a + list_sum xs
    apply(subst add.commute)
    ..
  also have ... = acc + (a + list_sum xs)
    apply(subst add.assoc[symmetric])
    ..
  also have ... = acc + list_sum (a#xs)
    apply(subst list_sum.simps(2))
    ..
  finally show ?case
    .
qed

lemma aux: fold (+) xs acc = acc + list_sum xs
proof(induction xs arbitrary: acc)
  — Style: declarative (Isar) skeleton, each case closed by automation (auto, new): shortest, but the individual reasoning steps are hidden inside auto.
  case Nil
  show ?case
    by auto
    — by m proves the goal in one step with method m, which must solve it completely.
    — auto tries to solve all goals automatically, combining simplification with logical reasoning.
next
  case (Cons x xs)
  show ?case
    using Cons.IH
    by auto
qed


lemma list_sum xs = list_sum' xs

  by (simp add: aux list_sum'_def)
  — simp rewrites the goal with simplification rules such as function equations; add: supplies extra rules, here aux and the definition of list_sum'.
  — Style: declarative one-step proof: simp with extra lemmas and a definition (new).
```

Exercise 3.2 Tail recursive *reverse*

In Isabelle, there is the reverse function *rev*, which, given a list, computes another list with the same elements but in reverse order. Take a loot into its definition:

thm *rev.simps*

Recall tail recursion: a function is tail recursive if in all recursive calls it does not call anything after itself[1]. For instance, the implementaion of *rev* in Isabelle is not tail recursive, because it calls (@) after it calls *rev* in the second equation.

Write an alternative implementation of the reverse function that is tail recursive.

Hint: recall that you can most of the times derive a tail recursive function using an accumulator argument.

```
fun rev'::'a list ⇒ 'a list ⇒ 'a list where
  rev' [] acc = acc
| rev' (x #xs) acc = rev' xs (x # acc)
```

```
fun rev_tr::'a list ⇒ 'a list where

  rev_tr xs = rev' xs []
```

Show that the two implementations are equivalent.

```
lemma rev'_append: rev' xs ys = (rev' xs []) @ ys
proof (induction xs arbitrary: ys)
  — Style: declarative (Isar); generalised list induction (arbitrary); subst with the IH; auto/simp.
  case Nil
  then show ?case
    — then passes the facts just obtained (here those of the current case) to the next statement,
like using.
    by auto
next
  case (Cons a xs)
  have 1: rev' xs (a # ys) = (rev' xs []) @ (a # ys)
    apply(subst Cons)
    ..
  have 2: rev' xs [a] = (rev' xs []) @ [a]
    apply(subst Cons)
    ..
  show ?case
    using 1 2
   by simp
qed
```

[1] See https://en.wikipedia.org/wiki/Tail_call. for a quick review.

lemma *rev_tr xs = rev xs*

proof(*induction xs*)
  — Style: declarative (Isar); list induction; equational reasoning with *simp*, *subst* and *auto*.
  case *Nil*
  then show *?case*
    by *auto*
next
  case (*Cons a xs*)
  have *rev_tr* (*a # xs*) = *rev' xs* [*a*]
    by *simp*
  also have ... = *rev' xs* [] @ [*a*]
    apply(*subst rev'_append*)
    ..
  also have ... = *rev* (*a # xs*)
    using *Cons.IH*
    by *auto*
  finally show *?case*
    .
qed

end